

Code Walkthrough Guide

Every code block in all 7 notebooks, explained line-by-line

Written so you can explain your code confidently, or rewrite it yourself from scratch if asked.

How to use this document

For each notebook, every code cell is shown in full, followed by a plain-English explanation of **what it does** and **why it was written that way**. If your supervisor asks "why did you do it like this?", the answer is right below the code.

Notebook 01 — Data Cleaning & Exploratory Data Analysis

Cell: Imports

```
python

import pandas as pd
import math
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
```

- `pandas` — the core library for loading and manipulating tabular data (your CSV becomes a `DataFrame`).
- `math` — only used later for `math.ceil()`, to calculate how many rows a plot grid needs.
- `matplotlib.pyplot` — the base plotting engine; `seaborn` sits on top of it and gives nicer-looking statistical plots (heatmaps, boxplots) with less code.
- `seasonal_decompose` — imported for potential trend/seasonal decomposition, though the main seasonality analysis is later established through Zivot-Andrews and ACF/PACF instead.

Cell: Check working directory

```
python

import os
print(os.getcwd())
```

Why: relative file paths (like `"../data/dataset_final/full_data.csv"`) are resolved *relative to wherever Python is currently running from* — not relative to where the notebook file physically sits on disk. Printing the current working directory is a quick sanity check before trying a relative path, so you know what "relative" actually means in that session.

Cell: Load the data

```
python

df = pd.read_csv("../data/dataset_final/full_data.csv")
```

Loads the raw CSV into a `DataFrame`. At this point, every column — including the date — is still just plain text/numbers; nothing has been converted to proper types yet.

Cell: Custom plotting functions

```
python

def heatmap(df, figsize=(12, 8), cmap='coolwarm', annot=True):
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns
    plt.figure(figsize=figsize)
    sns.heatmap(df[num_cols].corr(), annot=annot, cmap=cmap)
    plt.title('Correlation Matrix Heatmap', fontsize=16)
    plt.xlabel('Features', fontsize=12, labelpad=10)
    plt.ylabel('Features', fontsize=12)
    plt.show()
```

- `df.select_dtypes(include=['float64', 'int64'])` automatically picks out only the numeric columns — this means the function works safely even if non-numeric columns (like a leftover text date column) are still present; it won't crash trying to correlate text.

- `.corr()` computes the Pearson correlation coefficient between every pair of numeric columns, producing a square matrix.
- `sns.heatmap(...)` draws that matrix as a coloured grid, with `annot=True` printing the actual number inside each cell so it's readable without needing a separate table.

python

```
def hist(df, figsize=(12, 8), bins=50, color='blue', alpha=0.7):
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns
    df[num_cols].hist(figsize=figsize, bins=bins, color=color, alpha=alpha)
    plt.suptitle('Histograms of Numerical Features', fontsize=16)
    plt.show()
```

`.hist()` is a built-in pandas shortcut that draws one histogram per numeric column automatically. `bins=50` controls how many bars the value range is split into — more bins = finer detail, fewer bins = smoother/coarser shape.

python

```
def line(df, num_cols, ncols=2, figsize_width=14):
    nrows = math.ceil(len(num_cols) / ncols)
    fig, axes = plt.subplots(nrows, ncols, figsize=(figsize_width, 4 * nrows))
    axes = axes.flatten()
    for ax, col in zip(axes, num_cols):
        ax.plot(df.index, df[col], label=col)
        ax.set_title(f"{col} over time")
        ax.set_xlabel("Date")
        ax.set_ylabel(col)
        ax.grid(True, alpha=0.3)
    for ax in axes[len(num_cols):]:
        ax.remove()
    plt.tight_layout()
    plt.show()
```

- `math.ceil(len(num_cols) / ncols)` works out how many rows of subplots are needed, given you want `ncols` columns per row (round up, since a partial row still needs a full row of space). With 6 columns and `ncols=2`, that's `ceil(6/2) = 3` rows.
- `plt.subplots(nrows, ncols, ...)` creates a whole grid of empty subplot axes in one call.
- `axes.flatten()` turns the grid (which is normally a 2D array of axes, e.g. 3 rows × 2 columns) into a simple 1D list, so you can loop through it with a single `for` loop instead of nested loops.
- `zip(axes, num_cols)` pairs each subplot slot with a column name, one at a time — this is what lets one loop draw 6 different charts onto 6 different slots.
- The final loop (`axes[len(num_cols):]`) removes any *leftover empty* subplot slots — relevant if the grid has more slots than columns to fill (e.g. 7 columns in a 2-per-row grid would leave one empty slot in the last row).

python

```
def scatter(df, x_col, y_col, figsize=(12, 8), color='blue', alpha=0.7):
    plt.figure(figsize=figsize)
    plt.scatter(df[x_col], df[y_col], color=color, alpha=alpha)
    plt.title(f'Scatter Plot of {y_col} vs {x_col}', fontsize=16)
    plt.xlabel(x_col, fontsize=12)
    plt.ylabel(y_col, fontsize=12)
    plt.grid(True)
    plt.show()
```

A straightforward single scatter plot between two named columns — `alpha=0.7` makes points slightly see-through, which helps reveal overlapping points as darker patches when you have many observations stacked close together.

python

```
def boxplot(df, x_col, y_col, figsize=(12, 8), color='blue'):
    plt.figure(figsize=figsize)
    sns.boxplot(x=df[x_col], y=df[y_col], color=color)
    ...
```

Draws a boxplot of `y_col` grouped by the categories in `x_col`. (Note: this function was written for a categorical-x, numeric-y use case — your actual dataset has no categorical column, so where you used boxplots later, you called `sns.boxplot()` directly on all numeric columns side-by-side instead, bypassing this function.)

Cells: Initial inspection

```
python

df.head()      # first 5 rows
df.info()      # column types + non-null counts
df.describe()  # summary statistics (mean, std, min, max, quartiles)
df.dtypes      # just the data types
```

These four are standard "get to know your data" commands. `.info()` is particularly useful early on because it shows you, in one glance, whether any column has fewer non-null values than the total row count (a quick missing-data check) and whether a column that should be numeric is accidentally stored as text (`object` dtype).

Cell: Parse the date column

```
python

df['date'] = pd.to_datetime(df['month'], format='%YM%m')
```

Your raw `month` column looks like `"2015M07"` — a custom text format pandas can't guess automatically. `format='%YM%m'` spells out exactly how to read it:

- `%Y` = a 4-digit year (`2015`)
- `M` = the literal letter "M" that appears in the text (not a format code — just matched as-is)
- `%m` = a 2-digit month (`07`)

This produces a proper `Timestamp` object, defaulting to the 1st of that month (since no day information exists in the original text — this is just a technical placeholder and doesn't affect any time-series calculations, which only care about the monthly sequence and spacing).

Cells: Set the date as the index

```
python

df.set_index('date', inplace=True)
df.index.freq = 'MS'
```

- `set_index('date')` moves the `date` column from being a regular column into being the DataFrame's row label — required by virtually every time-series function used in later notebooks (ADF, ARIMA, plotting against time, etc.).
- `df.index.freq = 'MS'` explicitly tells pandas "this index represents Month-Start-spaced observations." This only succeeds if your dates are genuinely evenly spaced with no gaps — so setting this line is itself a built-in gap-check: if there were a missing month, this line would throw an error.

Cell: Check for missing values

```
python

df.isnull().sum()
```

Counts, per column, how many `NaN` (missing) values exist. Confirmed zero across the board for your dataset.

Cell: Correlation matrix (numbers) and heatmap (visual)

```
python

num_cols = df.select_dtypes(include=['float64', 'int64']).columns
df[num_cols].corr()
```

```
python

heatmap(df)
```

The first line explicitly builds the list of numeric column names once, so it can be reused throughout the rest of the notebook without re-typing all 6 names every time. The second line calls your custom function to visualise the same matrix.

Cell: Histograms, one per variable

```
python

for i in num_cols:
    hist(df[[i]], figsize=(8, 6), bins=50, color='blue', alpha=0.7)
```

Loops through each column name and calls `hist()` on a *single-column* DataFrame at a time (`df[[i]]` — double brackets keep it as a DataFrame rather than reducing it to a Series). This is why you got 6 separate histogram images rather than one combined grid — each loop iteration produces its own standalone figure.

Cell: A quick messy trend plot (superseded)

```
python

for i in num_cols:
    plt.plot(df.index, df[i])
```

An early, rougher attempt — this overlays *all six* series onto the *same single chart* with no labels or separate y-axis scales, which is hard to read since diesel/petrol/LPG (roughly 0–20) and CPI (roughly 50–270) sit on wildly different scales. This is superseded by the next cell's cleaner grid version.

Cell: The proper 6-panel trend grid

```
python

line(df, num_cols)
```

This one call produces the clean, readable 6-subplot grid (Figure 4.1) — each variable gets its own chart with its own appropriately-scaled y-axis, using the `line()` function defined earlier.

Cell: Pairplot

```
python

sns.pairplot(
    df[num_cols],
    diag_kind='hist',
    plot_kws={'alpha': 0.5, 's': 30, 'edgecolor': 'k'},
    height=3
)
```

`sns.pairplot` automatically builds a full grid of scatter plots for every pair of variables, with a histogram (`diag_kind='hist'`) on the diagonal for each variable's own distribution. `plot_kws` customises the scatter points themselves — `alpha=0.5` (semi-transparent), `s=30` (point size), `edgecolor='k'` (black outline on each point, making individual points easier to distinguish from overlapping neighbours).

Cell: Boxplot of all variables

```
python

plt.figure(figsize=(12, 10))
sns.boxplot(data=df[num_cols], palette="Set3")
plt.xticks(rotation=45, ha="right")
plt.title("Boxplots of Numerical Variables")
plt.tight_layout()
plt.show()
```

Draws all 6 variables' boxplots side-by-side in one chart (rather than using the earlier `boxplot()` function, which expected a categorical grouping column you don't have). `plt.xticks(rotation=45, ha="right")` angles the column-name labels on the x-axis so long names don't overlap each other.

Cell: All pairwise scatter plots in one grid

```
python

from itertools import combinations

def scatter_grid(df, num_cols, ncols=2, figsize_width=15, color='blue', alpha=0.6):
    pairs = list(combinations(num_cols, 2))
    nrows = math.ceil(len(pairs) / ncols)
    fig, axes = plt.subplots(nrows, ncols, figsize=(figsize_width, 4 * nrows))
    axes = axes.flatten()
    for ax, (x_col, y_col) in zip(axes, pairs):
        ax.scatter(df[x_col], df[y_col], color=color, alpha=alpha)
        ax.set_title(f"{y_col} vs {x_col}", fontsize=10)
        ax.set_xlabel(x_col, fontsize=8)
        ax.set_ylabel(y_col, fontsize=8)
        ax.grid(True, alpha=0.3)
    for ax in axes[len(pairs):]:
        ax.remove()
    plt.tight_layout()
    plt.show()
```

- `itertools.combinations(num_cols, 2)` generates every *unique* pair of column names, without repeating a pair in reverse order (so you get `(diesel, brent)` once, not also `(brent, diesel)` separately, since that would just be the same relationship viewed with axes swapped). With 6 columns, this produces `6 choose 2 = 15` pairs.
- The rest of the function follows exactly the same subplot-grid pattern as `line()` above: work out how many rows are needed, flatten the grid into a simple list, loop through pairs and slots together with `zip()`, then clean up unused slots.

Notebook 02 — Stationarity Testing

Cell: Imports

```
python

import pandas as pd
from statsmodels.tsa.stattools import adfuller, kpss, zivot_andrews
import warnings
warnings.filterwarnings('ignore')
```

`adfuller`, `kpss`, and `zivot_andrews` are the three statistical test functions from `statsmodels`. `warnings.filterwarnings('ignore')` suppresses non-critical warning messages these functions sometimes print (e.g. about the number of lags chosen) — this keeps notebook output clean, though it's worth knowing this line exists so you don't miss a *genuinely* important warning if something goes wrong.

Cell: Load and prepare data

```
python

df = pd.read_csv("../data/dataset_final/full_data.csv")
df['date'] = pd.to_datetime(df['month'], format='%YM%m')
df.set_index('date', inplace=True)
df.index.freq = 'MS'
num_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

Identical setup pattern to Notebook 01 — each notebook reloads and re-prepares the data independently, rather than depending on variables carried over from another notebook's memory. This is intentional: it makes each notebook runnable on its own, in any order, without needing to first run a different notebook.

Cell: The two testing functions

```
python
```

```
def stationarity_tests(series, name):
    """
    Runs ADF, KPSS, and Zivot-Andrews tests on a series at level.
    """
    results = {"series": name}

    adf_stat, adf_p, *_ = adfuller(series, autolag="AIC")
    results["ADF_stat"] = round(adf_stat, 4)
    results["ADF_pvalue"] = round(adf_p, 4)
    results["ADF_conclusion"] = "Stationary" if adf_p < 0.05 else "Non-stationary"

    kpss_stat, kpss_p, *_ = kpss(series, regression="c", nlags="auto")
    results["KPSS_stat"] = round(kpss_stat, 4)
    results["KPSS_pvalue"] = round(kpss_p, 4)
    results["KPSS_conclusion"] = "Non-stationary" if kpss_p < 0.05 else "Stationary"

    za_stat, za_p, _, _, za_break = zivot_andrews(series, regression="c")
    results["ZA_stat"] = round(za_stat, 4)
    results["ZA_pvalue"] = round(za_p, 4)
    results["ZA_break_date"] = series.index[za_break].strftime("%Y-%m")
    results["ZA_conclusion"] = "Stationary (with break)" if za_p < 0.05 else "Non-stationary"

    return results
```

- `adfuller(series, autolag="AIC")` runs the ADF test. `autolag="AIC"` tells the function to automatically choose how many lagged difference terms to include in the test regression, using AIC to pick the best number — you don't have to guess this by hand.
- `adfuller(...)` returns several values, but the function only needs the first two (`adf_stat`, `adf_p`); `*` is Python syntax meaning "throw away everything else the function returned that I'm not using."
- The conclusion logic directly encodes the rule explained in the paper: **for ADF, a small p-value means stationary** (`adf_p < 0.05` → "Stationary"), because ADF's null hypothesis is non-stationarity.
- `kpss(series, regression="c", nlags="auto")` runs KPSS. `regression="c"` means "test around a constant mean" (as opposed to `"ct"`), which would test around a constant plus a trend line — you're testing whether the series is stable around one flat average, which is the standard choice here).
- The KPSS conclusion logic is **flipped** relative to ADF: **a small p-value means non-stationary** (`kpss_p < 0.05` → "Non-stationary"), because KPSS's null hypothesis is stationarity — the opposite starting assumption from ADF.
- `zivot_andrews(series, regression="c")` returns, among other things, `za_break` — the *index position* (not the actual date) of the most likely structural break point the test found. `series.index[za_break]` converts that position back into an actual date, and `.strftime("%Y-%m")` formats it as readable text like `"2022-01"`.

python

```
def stationarity_tests_diff(series, name):
    """Runs ADF and KPSS on an already-differenced series."""
    ...
```

A slightly trimmed-down version of the same function, used after differencing — it skips the Zivot-Andrews break test, since ZA's purpose (finding a break in an otherwise-stable series) is specifically a level-data question; once you've already differenced the series to handle a structural break's effect on the trend, re-running ZA on the differenced series isn't part of the standard workflow here.

Cell: Running the tests at each stage

python

```
level_results = [stationarity_tests(df[col], col) for col in num_cols]
level_results_df = pd.DataFrame(level_results)
```

A **list comprehension**: this is shorthand for a `for` loop that builds a list. It's exactly equivalent to:

python

```
level_results = []
for col in num_cols:
    level_results.append(stationarity_tests(df[col], col))
```

but written on one line. Each call runs the full three-test suite on one column and returns a dictionary of results; the list of dictionaries is then converted into a proper table with `pd.DataFrame(...)`.

```
python

df_diff1 = df[num_cols].diff().dropna()
```

`.diff()` computes, for every column simultaneously, the difference between each row and the row before it (this month minus last month). The very first row has nothing before it to subtract, so it becomes `NaN` — `.dropna()` removes that one now-empty row, leaving 131 rows instead of 132.

```
python

var_cols = ["diesel_price_(GHC/litre)", "petrol_price_ghs/litre", "lpg_ghs/kg",
            "brent_price_usd_per_barrel", "usd/ghs_rate", "consumer_price_index"]
df_diff2 = df[var_cols].diff().diff().dropna()
```

Chaining `.diff().diff()` applies the differencing operation twice in a row — first difference, then difference *that* result again. Two rows are lost this time (130 remain), since each differencing step removes one more leading row.

Notebook 03 — Model Identification (ACF/PACF)

Full cell

```
python

import warnings
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

warnings.filterwarnings("ignore")

df = pd.read_csv("../data/dataset_final/full_data.csv")
df["date"] = pd.to_datetime(df["month"], format="%Y%m")
df = df.set_index("date").asfreq("MS")

fuel_cols = ["diesel_price_(GHC/litre)", "petrol_price_ghs/litre", "lpg_ghs/kg"]
external_cols = ["brent_price_usd_per_barrel", "usd/ghs_rate", "consumer_price_index"]

diff1 = df[fuel_cols + external_cols].diff().dropna()
print(f"Observations: {len(df)} ({df.index.min():%Y-%m} to {df.index.max():%Y-%m})")
print("Missing values after differencing:")
print(diff1.isna().sum())

for column in fuel_cols:
    fig, axes = plt.subplots(1, 2, figsize=(14, 4))
    plot_acf(diff1[column], lags=min(24, len(diff1) // 2 - 1), ax=axes[0], alpha=0.05)
    plot_pacf(diff1[column], lags=min(24, len(diff1) // 2 - 1), ax=axes[1], alpha=0.05, method="ywm")
    axes[0].set_title(f"ACF of first difference: {column}")
    axes[1].set_title(f"PACF of first difference: {column}")
    plt.tight_layout()
    plt.show()
```

New things worth explaining here:

- `df.set_index("date").asfreq("MS")` — a slightly more compact one-line way of doing what Notebook 01 did in two separate steps (`set_index` then setting `.freq()`). `.asfreq("MS")` both sets the index and enforces/validates the monthly frequency in one call.
- `lags=min(24, len(diff1) // 2 - 1)` — a safety cap. Statistically, you should never look at more lags than roughly half your sample size (a common rule of thumb), otherwise the plot becomes unreliable at the far right-hand end. `//` is Python's *integer* (floor)

division — it discards any remainder, so `131 // 2` gives `65`, not `65.5`. Taking `min(24, ...)` means "use 24 lags, unless half the sample would give you fewer than that, in which case use the smaller number" — a defensive cap that keeps the plot readable and prevents a crash on a very short series.

- `plot_acf(..., alpha=0.05)` — this `alpha` isn't transparency (like in the scatter functions) — here it's the *significance level* for the shaded confidence band, i.e. it draws the 95% confidence interval ($100\% - 5\% = 95\%$).
- `method="ywm"` in `plot_pacf` — this selects the "Yule-Walker with correction" method for calculating the partial autocorrelation values, a numerically stable, commonly recommended estimation method for PACF.

Notebook 04 — ARIMA Modeling

Cell 1: Setup

```
python

import warnings
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.arima.model import ARIMA

warnings.filterwarnings("ignore")

df = pd.read_csv("../data/dataset_final/full_data.csv")
df["date"] = pd.to_datetime(df["month"], format="%Y%m")
df = df.set_index("date").asfreq("MS")
fuel_cols = ["diesel_price_(GHC/litre)", "petrol_price_ghs/litre", "lpg_ghs/kg"]
horizon = 6
train = df.iloc[:-horizon]
test = df.iloc[-horizon:]

print(f"Training period: {train.index.min():%Y-%m} to {train.index.max():%Y-%m}")
print(f"Test period: {test.index.min():%Y-%m} to {test.index.max():%Y-%m}")
```

New things here:

- `sklearn.metrics.mean_absolute_error, mean_squared_error` — ready-made functions for MAE and RMSE, rather than writing the averaging formulas by hand (RMSE is the square root of `mean_squared_error`, computed separately below).
- `acorr_ljungbox` — the Ljung-Box test function.
- `ARIMA` — the actual model-fitting class from `statsmodels`.
- `horizon = 6` then `train = df.iloc[:-horizon]` / `test = df.iloc[-horizon:]` — this is the train/test split. `df.iloc[:-horizon]` means "every row except the last 6"; `df.iloc[-horizon:]` means "just the last 6 rows." This single pair of lines is what creates your 126-month training set and 6-month test set.

Cell 2: Order selection

```
python

candidate_orders = [(p, 1, q) for p in range(4) for q in range(4)]
```

Another list comprehension, this time with **two nested loops in one line** — equivalent to:

```
python

candidate_orders = []
for p in range(4):
    for q in range(4):
        candidate_orders.append((p, 1, q))
```

`range(4)` produces `(0, 1, 2, 3)`. So this builds all 16 combinations of `p` and `q` (each from 0-3), always with `d=1` fixed (since Notebook 02 established first-differencing as correct for all three fuels).

python

```
for column in fuel_cols:
    differenced = train[column].diff().dropna()
    fig, axes = plt.subplots(1, 2, figsize=(14, 4))
    plot_acf(differenced, lags=min(36, len(differenced) // 2 - 1), ax=axes[0])
    plot_pacf(differenced, lags=min(36, len(differenced) // 2 - 1), ax=axes[1], method="ywm")
    ...
```

Re-runs the same ACF/PACF visual check from Notebook 03, but specifically **on the training set only** (126 months, not the full 132) — this matters because order selection must only ever look at training data, never at the held-out test period, or the whole point of the test set is undermined.

python

```
best_order = None
best_aic = float("inf")
for order in candidate_orders:
    try:
        fit = ARIMA(train[column], order=order, enforce_stationarity=False).fit()
        order_rows.append({"series": column, "order": order, "AIC": fit.aic, "BIC": fit.bic})
        if fit.aic < best_aic:
            best_order, best_aic = order, fit.aic
    except (ValueError, np.linalg.LinAlgError):
        continue
selected_orders[column] = best_order
```

This is the actual AIC grid search:

- `best_aic = float("inf")` starts the "best so far" score at infinity, so that literally any real result will immediately beat it on the first comparison.
- The `for order in candidate_orders` loop tries every one of the 16 (p,1,q) combinations in turn, fitting a full ARIMA model each time.
- `enforce_stationarity=False` tells `statsmodels` not to reject a fitted model just because its estimated AR coefficients technically imply near-non-stationary behaviour — this is a practical relaxation that allows the search to complete even for edge-case parameter combinations, rather than crashing.
- `try / except (ValueError, np.linalg.LinAlgError): continue` — some (p,q) combinations can fail to fit (e.g. numerical instability, or too many parameters for too little data) and would otherwise crash the whole loop. This catches those specific error types and simply skips to the next candidate (`continue`) instead of stopping everything.
- Every attempt's result is logged to `order_rows` (for the full comparison table), while `best_order`/`best_aic` separately track only the single winning combination so far.
- `selected_orders[column] = best_order` stores the final winning (p,1,q) for this fuel into a dictionary, so it can be reused in the next cell.

Cell 3: Fit, diagnose, forecast

python

```
for column, order in selected_orders.items():
    fit = ARIMA(train[column], order=order, enforce_stationarity=False).fit()
    forecast = fit.forecast(steps=horizon)
    arima_forecasts[column] = forecast
```

`selected_orders.items()` loops through the dictionary built in the previous cell, giving you both the column name *and* its winning order together in each iteration. `.fit()` actually estimates the model's coefficients on the training data. `.forecast(steps=horizon)` produces exactly 6 forecasted values, one for each month of the held-out test period.

python

```
residuals = fit.resid.dropna()
ljung_box = acorr_ljungbox(residuals, lags=[10], return_df=True)
```

`fit.resid` gives you the model's *in-sample* errors — the difference between what the model predicted and what actually happened, for every month in the training period (this is different from `forecast`, which is *out-of-sample*, i.e. genuinely unseen future months). `acorr_ljungbox(..., lags=[10], return_df=True)` runs the Ljung-Box test checking for leftover pattern up to 10 lags combined, and returns the result as a small table (`return_df=True`) rather than a plain tuple, which is easier to read and extract from.

```
python

fit.plot_diagnostics(figsize=(12, 8))
```

A single built-in `statsmodels` method that automatically produces all four residual diagnostic panels (standardized residuals, histogram+KDE, Q-Q plot, correlogram) in one call — this is why every diagnostic image in your chapter has exactly the same 2×2 layout: it's the library's own standard output, not something built manually.

```
python

errors = test[column].to_numpy() - forecast.to_numpy()
arma_metrics.append({
    "model": "ARIMA", "series": column, "order": str(order),
    "MAE": mean_absolute_error(test[column], forecast),
    "RMSE": np.sqrt(mean_squared_error(test[column], forecast)),
    "MAPE": np.mean(np.abs(errors / test[column].to_numpy())) * 100,
})
```

- `.to_numpy()` converts a pandas Series into a plain NumPy array — needed here so the subtraction lines up by *position* rather than trying to match by date-index labels (both arrays are already the same length and order, so this is safe and slightly faster).
- `errors = actual - forecast` — the raw forecast errors, one per test month.
- MAE and RMSE come straight from `sklearn`'s ready-made functions.
- MAPE isn't built into `sklearn` the same way, so it's computed manually: `errors / actual` gives the error as a *fraction* of the true value for each month, `np.abs(...)` makes every fraction positive (so overestimates and underestimates don't cancel out), `np.mean(...)` averages across the 6 months, and `* 100` converts the fraction into a percentage.

```
python

for column in fuel_cols:
    plt.figure(figsize=(12, 4))
    plt.plot(train.index, train[column], label="Train")
    plt.plot(test.index, test[column], label="Observed", color="black")
    plt.plot(arma_forecasts.index, arma_forecasts[column], label="ARIMA forecast")
    plt.title(f"ARIMA forecast: {column}")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

The final forecast-vs-actual plots — three separate line traces on the same chart: the training history (blue, by default), the real observed test-period values (black, so it stands out clearly), and the model's forecast (a third colour) — letting you visually see how close the forecast landed to reality.

Notebook 05 — ARIMAX Modeling

The structure is **almost identical to Notebook 04**, so only the genuinely new parts are explained here.

```
python

external_cols = ["brent_price_usd_per_barrel", "usd/ghs_rate"]
```

The two variables that will be given to the model as extra predictive information.

```
python

fit = ARIMA(train[column], exog=train[external_cols], order=order, enforce_stationarity=False).fit()
```

The single, crucial difference from ARIMA: the `exog=train[external_cols]` argument. This tells `statsmodels` "in addition to modeling this series' own past values and errors, also use these extra columns as predictors at each time point."

```
python

forecast = fit.forecast(steps=horizon, exog=test[external_cols])
```

Critically, forecasting with an exogenous model requires supplying the exogenous values *for the future period being forecasted too* — `exog=test[external_cols]` hands the model the actual real Brent/exchange-rate values that occurred during the test months. This is exactly the "conditional forecast" caveat explained in the results document: the model isn't predicting Brent and the exchange rate itself, it's assuming you already know them.

Everything else — the order search loop, Ljung-Box, `plot_diagnostics()`, the metrics calculations, the final forecast plot — is structurally the same code pattern as Notebook 04, just with `exog=` added in the two places where it matters (fitting and forecasting).

Notebook 06 — Rolling-Origin Forecast Evaluation

This is the most structurally complex notebook. Breaking it into pieces:

Setup and fixed order selection

```
python

var_cols = fuel_cols + external_cols + ["consumer_price_index"]
max_horizon = 6
initial_train_size = 96
candidate_orders = [(p, 1, q) for p in range(4) for q in range(4)]

initial_train = df.iloc[:initial_train_size]

def select_arima_order(series, exog=None):
    best_order = None
    best_aic = float("inf")
    for order in candidate_orders:
        try:
            fit = ARIMA(series, exog=exog, order=order, enforce_stationarity=False).fit()
            if fit.aic < best_aic:
                best_order, best_aic = order, fit.aic
        except (ValueError, np.linalg.LinAlgError):
            continue
    return best_order
```

This wraps the same AIC-search logic from Notebook 04 into a **reusable function** — `select_arima_order()` — that can optionally take an `exog=` argument, meaning the *exact same function* is used for both the plain ARIMA search and the ARIMAX search, just by passing (or not passing) exogenous data:

```
python

arima_orders = {column: select_arima_order(initial_train[column]) for column in fuel_cols}
arimax_orders = {column: select_arima_order(initial_train[column], initial_train[external_cols]) for column in fuel_cols}
```

Both order sets are selected using only the **first 96 months** (`initial_train`) — this is the "identify orders once" step described in the results document, done before any of the 31 repeated rolling tests begin.

VAR lag selection for the rolling test

```
python
```

```

initial_var_diff = initial_train[var_cols].diff().diff().dropna()
var_model = VAR(initial_var_diff)
lag_selection = var_model.select_order(maxlags=6)
stable_var_candidates = []
for lag in range(1, 7):
    try:
        candidate_fit = var_model.fit(lag)
        if candidate_fit.is_stable():
            stable_var_candidates.append((candidate_fit.aic, lag))
    except (ValueError, np.linalg.LinAlgError):
        continue

if stable_var_candidates:
    selected_var_lag = min(stable_var_candidates)[1]
else:
    selected_var_lag = max(1, min(lag_selection.selected_orders.get("bic") or 1, 6))

```

- `.is_stable()` is a built-in check confirming the VAR system's dynamics won't spiral off to unrealistic values — without this check, a VAR could be mathematically valid to fit but useless for forecasting.
- `stable_var_candidates` collects `(aic, lag)` pairs, but *only* for lags that passed the stability check.
- `min(stable_var_candidates)[1]` — because each entry is a tuple `(aic, lag)`, Python's `min()` on a list of tuples compares them starting with the *first* element of each tuple by default — so this automatically finds the tuple with the lowest AIC, and `[1]` then pulls out just the lag number from that winning tuple.
- The `else` branch is a fallback safety net: if literally *no* lag was stable (unlikely, but possible), fall back to whatever BIC would have suggested, capped between 1 and 6, rather than letting the whole notebook crash with no VAR lag chosen at all.

The rolling loop itself

```

python

errors = {
    model_name: {column: {horizon: [] for horizon in range(1, max_horizon + 1)} for column in fuel_cols}
    for model_name in ["ARIMA", "ARIMAX", "VAR"]}

```

This builds a **nested dictionary** three levels deep — `errors[model_name][column][horizon]` — each pointing to an empty list that will collect forecast errors as the rolling loop runs. It's built with three nested dictionary comprehensions, read from the outside in: for each model name, build a dictionary of columns; for each column, build a dictionary of horizons; for each horizon, start with an empty list.

```

python

origins = range(initial_train_size, len(df) - max_horizon + 1)
for origin in origins:
    history = df.iloc[:origin]
    future = df.iloc[origin : origin + max_horizon]

```

`origins` is the sequence of starting points to test from: beginning at month 96, and stopping early enough that there's always at least 6 real future months left to compare against `(len(df) - max_horizon + 1)`. For each origin, `history` is everything known up to that point, and `future` is the next 6 actual months — used as the "correct answers" to score the forecast against.

```

python

```

```

for column in fuel_cols:
    arima_fit = ARIMA(history[column], order=arima_orders[column], enforce_stationarity=False).fit()
    arima_forecast = arima_fit.forecast(steps=max_horizon)

    arimax_fit = ARIMA(history[column], exog=history[external_cols], order=arimax_orders[column], enforce_stationarity=False)
    arimax_forecast = arimax_fit.forecast(steps=max_horizon, exog=future[external_cols])

    for horizon in range(1, max_horizon + 1):
        actual = future[column].iloc[horizon - 1]
        errors["ARIMA"][column][horizon].append(actual - arima_forecast.iloc[horizon - 1])
        errors["ARIMAX"][column][horizon].append(actual - arimax_forecast.iloc[horizon - 1])

```

At every single origin, brand-new ARIMA and ARIMAX models are fitted **from scratch** on whatever history is available up to that point (using the *fixed* orders selected earlier — not re-searching AIC every time, which is the computational-cost simplification mentioned in the results). `.iloc[horizon - 1]` is needed because `horizon` counts from 1 (1-month-ahead, 2-months-ahead...) but Python list/array positions count from 0 — so "1 month ahead" is actually stored at position 0.

python

```

history_diff = history[var_cols].diff().diff().dropna()
var_fit = VAR(history_diff).fit(selected_var_lag)
var_diff_forecast = var_fit.forecast(history_diff.values[-selected_var_lag:], steps=max_horizon)

```

The VAR forecast at each origin: difference the current history twice, fit at the fixed lag, and forecast forward. `history_diff.values[-selected_var_lag:]` takes just the *last* `selected_var_lag` rows of the differenced history — VAR's `.forecast()` function needs exactly this many most-recent rows as its "starting point" to project forward from (this is analogous to needing the last few dominoes standing to know which way the next ones will fall).

python

```

last_level = history[var_cols].iloc[-1]
last_first_difference = history[var_cols].diff().dropna().iloc[-1]
var_first_difference_forecast = last_first_difference.to_numpy() + np.cumsum(var_diff_forecast, axis=0)
var_level_forecast = last_level.to_numpy() + np.cumsum(var_first_difference_forecast, axis=0)

```

This is the "undo the differencing" (integration) step explained conceptually in the results document. `np.cumsum(..., axis=0)` computes a **running total down each column** — so if `var_diff_forecast` holds 6 rows of forecasted second-differences, `np.cumsum` turns row 3, say, into "the sum of rows 1 through 3 so far." Adding `last_first_difference` (the last *known, real* first-difference value) to that running total reconstructs a forecast of the first differences; repeating the same cumulative-sum trick one level up, added to `last_level` (the last known real price), reconstructs the actual forecasted price level.

Turning errors into accuracy metrics

python

```

metric_rows = []
for model_name in errors:
    for column in fuel_cols:
        for horizon in range(1, max_horizon + 1):
            error = np.asarray(errors[model_name][column][horizon], dtype=float)
            actual = df[column].iloc[initial_train_size + horizon - 1 : len(df) - max_horizon + horizon].to_numpy()
            metric_rows.append({
                "model": model_name, "series": column, "horizon_months": horizon,
                "MAE": np.mean(np.abs(error)),
                "RMSE": np.sqrt(np.mean(error**2)),
                "MAPE": np.mean(np.abs(error / actual)) * 100,
                "origins": len(error),
            })

```

After the rolling loop finishes, `errors[model_name][column][horizon]` holds a list of 31 individual error values (one from each origin). This block loops through every model/column/horizon combination and collapses each list of 31 errors down into three summary numbers (MAE, RMSE, MAPE) — the same formulas as before, just applied here to 31 pooled values instead of 6.

The Diebold-Mariano test, from scratch

python

```
def diebold_mariano(errors_one, errors_two, horizon):
    loss_difference = np.asarray(errors_one) ** 2 - np.asarray(errors_two) ** 2
    sample_size = len(loss_difference)
    mean_difference = loss_difference.mean()
    centered = loss_difference - mean_difference
    bandwidth = min(max(horizon - 1, 0), sample_size - 1)
    variance = np.mean(centered**2)
    for lag in range(1, bandwidth + 1):
        weight = 1 - lag / (bandwidth + 1)
        covariance = np.mean(centered[lag:] * centered[:-lag])
        variance += 2 * weight * covariance
    variance = max(variance, np.finfo(float).eps)
    statistic = mean_difference / np.sqrt(variance / sample_size)
    p_value = 2 * norm.sf(abs(statistic))
    return statistic, p_value
```

This function is written from first principles rather than imported from a library, so it's worth understanding step by step:

1. `loss_difference = errors_one2 - errors_two2` — DM compares models using *squared* errors (a common loss function choice), and looks at the *difference* in loss between the two models, one forecast-origin at a time.
2. `mean_difference = loss_difference.mean()` — if Model 1 is genuinely more accurate, this average should be reliably negative (its squared errors tend to be smaller).
3. `centered = loss_difference - mean_difference` — subtracting the mean from every value ("centering" the data around zero) is a standard step before calculating variance.
4. **The HAC (Heteroskedasticity and Autocorrelation Consistent) variance calculation** — the `for lag in range(1, bandwidth+1)` loop: forecast errors at overlapping horizons (like 3-month-ahead forecasts made from origins one month apart) aren't independent of each other, so a naive variance calculation would understate the true uncertainty. This loop adds in "how correlated is this loss-difference with itself, a few steps ago" terms, weighted down (`weight = 1 - lag/(bandwidth+1)`) the further back you look — this weighting scheme is the standard Newey-West style correction.
5. `bandwidth = min(max(horizon - 1, 0), sample_size - 1)` — sets how many of those lagged correlation terms to include. This follows Diebold & Mariano's own original guidance: for an h -step-ahead forecast, use $h-1$ as the bandwidth (a 1-month-ahead forecast needs 0 extra lag terms; a 6-month-ahead forecast needs 5).
6. `statistic = mean_difference / sqrt(variance / sample_size)` — this is a standard "z-score" style test statistic: how many standard errors away from zero is the observed average loss difference?
7. `p_value = 2 * norm.sf(abs(statistic))` — converts that statistic into a p-value using the normal distribution. `norm.sf` ("survival function") gives the probability of getting a value *at least this extreme* in one direction; multiplying by 2 makes it a **two-tailed** test (checking for a difference in *either* direction — Model 1 better, or Model 2 better — not assuming in advance which one you expect to win).

Cumulative version (Cell 3)

The final cell repeats a very similar exercise, but instead of looking at each horizon separately, it **pools** errors across horizons 1 through a cutoff (1, 3, or 6) using `np.concatenate(...)` (joins multiple arrays end-to-end into one longer array), giving you the "cumulative" tables used in Chapter 4 (e.g. "average error across the first 3 months combined," rather than "error at exactly month 3").

Notebook 07 — Vector Autoregression (VAR)

Setup and differencing

python

```
var_cols = fuel_cols + ["brent_price_usd_per_barrel", "usd/ghs_rate", "consumer_price_index"]
train_diff2 = train[var_cols].diff().diff().dropna()
model = VAR(train_diff2)
```

`VAR(train_diff2)` doesn't fit anything yet — it just creates a VAR "model object" bound to this dataset, ready to be fit at whichever lag you choose next. This two-step pattern (create the model, then `.fit(lag)`) is common in `statsmodels` and lets you try multiple lags without reloading the data each time.

Lag selection and stability check

```
python

lag_selection = model.select_order(maxlags=6)
stable_candidates = []
for lag in range(1, 7):
    try:
        candidate_fit = model.fit(lag)
        if candidate_fit.is_stable():
            stable_candidates.append((candidate_fit.aic, lag, candidate_fit))
    except (ValueError, np.linalg.LinAlgError):
        continue

if stable_candidates:
    _, selected_lag, var_fit = min(stable_candidates, key=lambda item: item[0])
```

- `model.select_order(maxlags=6)` is a built-in `statsmodels` convenience function that automatically computes AIC, BIC, and a couple of other criteria for every lag from 1 to 6 and prints a summary table — this is what produces the "Lag-order selection" table showing AIC and BIC disagreeing.
- The manual loop below it exists because `select_order()` alone doesn't check *stability* — so the code re-fits every lag itself and explicitly filters for stability before picking a winner.
- `min(stable_candidates, key=lambda item: item[0])` — since each entry is a 3-part tuple `(aic, lag, fitted_model)`, `key=lambda item: item[0]` tells Python's `min()` function "when comparing these tuples, only look at the first element (the AIC) to decide which is smallest" — otherwise `min()` would try to compare the fitted model objects too, which doesn't make sense and would cause an error.
- `_, selected_lag, var_fit = ...` unpacks the winning tuple into three separate variables in one line; the underscore `_` is a Python convention meaning "I need this position in the tuple, but I'm deliberately not going to use this value, so I won't give it a real name."

Residual diagnostics

```
python

print(var_fit.test_whiteness(nlags=10))
print(var_fit.test_normality())
```

Two more built-in `statsmodels` methods — `test_whiteness()` is the multivariate (system-wide) equivalent of the Ljung-Box test, and `test_normality()` is the multivariate equivalent of a normality test, both bundled together for the whole 6-variable system at once rather than one variable at a time.

Granger causality loop

```
python

granger_rows = []
for target in fuel_cols:
    for source in var_cols:
        if source == target:
            continue
        causality = var_fit.test_causality(caused=target, causing=source, kind="f")
        granger_rows.append({
            "causing_variable": source, "target_variable": target,
            "test_statistic": causality.test_statistic,
            "p_value": causality.pvalue,
            "significant_at_5_percent": causality.pvalue < 0.05,
        })
```

Nested loop: for every fuel price (`target`), test every *other* variable in the system (`source`) as a possible cause — `if source == target: continue` skips testing a variable against itself, which wouldn't be a meaningful Granger test. `var_fit.test_causality(caused=..., causing=..., kind="f")` runs the actual test using an F-statistic (a standard test-statistic choice for this kind of joint-hypothesis test). With 3 fuel targets $\times$ 5 valid other variables each, this produces the 15-row Granger causality table.

Impulse response functions

```
python

irf = var_fit.irf(12)
irf_mean = irf.irfs
irf_lower, irf_upper = irf.errband_mc(orth=False, repl=500, signif=0.05)
```

- `var_fit.irf(12)` computes the impulse response for 12 months ahead.
- `irf.irfs` is a 3-dimensional array holding the estimated response of *every* variable to a shock in *every other* variable, at *everyone* of the 12 months — the code later slices out just the fuel-price-responding-to-external-shock combinations it actually wants to plot.
- `irf.errband_mc(orth=False, repl=500, signif=0.05)` computes uncertainty bands around those responses using **Monte Carlo simulation** — repeatedly (500 times, `repl=500`) simulating plausible alternative versions of the fitted model (based on the estimated uncertainty in the coefficients) and seeing how much the impulse response shape varies across those 500 simulations, then taking the 5th and 95th percentile of that spread as the lower/upper band (`signif=0.05` = 5% significance, i.e. a 95% band). `orth=False` means the shocks are **not orthogonalized** — a technical choice about whether to assume the shocks to different variables are statistically independent of each other; leaving it `False` here means the reported responses don't rely on that stronger assumption.

```
python

response_indices = [var_cols.index(column) for column in fuel_cols]
impulse_indices = [var_cols.index(column) for column in external_cols]
```

`var_cols.index(column)` finds *where* a given column name sits in the `var_cols` list (its numeric position) — needed because the big 3D `irf_mean` array is indexed by position number, not by column name, so this translates your human-readable column names into the numeric positions the array actually needs.

```
python

for row, response_index in enumerate(response_indices):
    for column, impulse_index in enumerate(impulse_indices):
        axis = axes[row, column]
        response = irf_mean[:, response_index, impulse_index]
        ...
```

`enumerate(...)` gives you both a position counter (`row`/`column`, used to pick the right subplot slot) and the actual value (`response_index`/`impulse_index`, used to slice the right data) at the same time. `irf_mean[:, response_index, impulse_index]` reads: "give me all 12 time-months (`:`), for this specific response variable, reacting to this specific shock variable" — pulling out one single response curve from the big 3D array.

Forecast-error variance decomposition

```
python

fevd = var_fit.fevd(6)
fevd_values = fevd.decomp[5, response_indices, :]
```

`var_fit.fevd(6)` computes the variance decomposition out to 6 steps ahead. `fevd.decomp` is again a 3D array (steps × variables × variables); `[5, response_indices, :]` reads as "give me step index 5 (which is the *6th* step, since counting starts at 0), for just the fuel-price rows (`response_indices`), across *all* shock-source columns (`:`)" — producing exactly the 3-row × 6-column table that gets plotted as the FEVD heatmap.

The final forecast and level reconstruction

```
python

lagged_values = train_diff2.values[-selected_lag:]
diff2_forecast = var_fit.forecast(y=lagged_values, steps=horizon)
```

Identical pattern to the rolling-origin VAR forecasting explained above — take the last `selected_lag` rows of the (twice-)differenced training data as the "launch point," and forecast forward 6 steps in twice-differenced units.

python

```
diff2_forecast = pd.DataFrame(diff2_forecast, index=test.index, columns=var_cols)
last_level = train[var_cols].iloc[-1]
last_first_difference = train[var_cols].diff().dropna().iloc[-1]
first_difference_forecast = pd.DataFrame(
    last_first_difference.to_numpy() + diff2_forecast.cumsum().to_numpy(),
    index=test.index, columns=var_cols)
level_forecast = pd.DataFrame(
    last_level.to_numpy() + first_difference_forecast.cumsum().to_numpy(),
    index=test.index, columns=var_cols)
```

The same two-step "integrate twice" logic explained in Notebook 06, just using pandas' `.cumsum()` method directly on a DataFrame (equivalent to `np.cumsum(..., axis=0)` used earlier) rather than the raw NumPy function — both do the same running-total calculation, just called slightly differently depending on whether you're working with a plain array or a labelled DataFrame at that point in the code.

A note on writing this yourself

If your supervisor wants you to demonstrate you can write this without relying on a saved copy, the parts most worth being able to reproduce **from memory, conceptually** (rather than remembering exact syntax) are:

1. The date-parsing and index-setup block (Notebook 01) — used identically at the top of every single notebook.
2. The AIC grid-search pattern (`for order in candidate_orders: try: fit... except: continue`) — this exact loop shape reappears in Notebooks 04, 05, and 06.
3. The "difference twice, forecast, then cumsum twice to get back to price levels" pattern for VAR — this is the trickiest piece mathematically, and appears in both Notebooks 06 and 07.
4. The Diebold-Mariano function — if asked to derive it, the key ideas to explain are: squared-error loss difference → HAC-adjusted variance (because forecast errors are autocorrelated) → z-score-style test statistic → two-tailed p-value.

Everything else (plotting calls, metric calculations, `.fit()`/`.forecast()`) follows very standard, repeatable patterns once those four core ideas are understood.